

CIENCIA DE DATOS (TA047)

Cátedra Dr. Ing. Rodríguez - 1°C 2025

Trabajo práctico 1 – Grupo 3

Ejercicio 3: Regresión

Integrantes:

- **Arias Rafaela Pilar**, Padrón: 112272,
Email: rparias@fi.uba.ar
- **Barcala Roca Ramiro**, Padrón: 112478,
Email: rbarcala@fi.uba.ar
- **Ienco, Lara Eliana**, Padrón: 111921,
Email: liencho@fi.uba.ar
- **Lema Mora**, Padrón: 111831,
Email: molema@fi.uba.ar

Fecha de entrega: 01/05/25

Ejercicio 3: Regresión

Breve descripción y análisis del dataset

Ciudad asignada: Río de Janeiro, Brasil.

El dataset contiene datos de alojamiento de la plataforma Airbnb. El objetivo es, mediante diversos modelos, predecir el precio de alquiler de una propiedad en función de los datos publicados en el aviso. La cantidad de registros inicial del Dataset (sin filtrar por la ciudad asignada) era de 39.499 filas y 75 columnas. Tras la filtración por la ciudad de Río de Janeiro, pasó a tener 24.442 filas y la misma cantidad de columnas.

Luego de un breve análisis teniendo en cuenta la descripción de las variables (disponible en el Colab ya que es muy extensa) concluimos en la siguiente categorización:

Tipo de variable	Cuantitativas	Tipo de variable	Cualitativas
Continua	last_scraped, host_since, host_response_rate, host_acceptance_rate, latitude, longitude, price, minimum_nights_avg_ntm, maximum_nights_avg_ntm, calendar_updated, calendar_last_scraped, first_review, last_review, review_scores_rating, review_scores_accuracy, review_scores_cleanliness, review_scores_checkin, review_scores_communication, review_scores_location, review_scores_value, reviews_per_month	Nominal	listing_url, source, name, description, neighborhood_overview, picture_url, host_url, host_name, host_location, host_is_superhost, host_thumbnail_url, host_picture_url, host_neighbourhood, host_verifications, host_has_profile_pic, host_identity_verified, neighbourhood, neighbourhood_cleansed, neighbourhood_group_cleansed, property_type, room_type, bathrooms_text, amenities, has_availability, license, instant_bookable, host_about
Discreta	id, scrape_id, host_id, host_listings_count, host_total_listings_count, accommodates, bathrooms, bedrooms, beds, minimum_nights, maximum_nights, minimum_minimum_nights, maximum_minimum_nights, minimum_maximum_nights,	Ordinal	host_response_time

	maximum_maximum_nights, availability_30, availability_60, availability_90, availability_365, number_of_reviews, number_of_reviews_ltm, number_of_reviews_l30d, calculated_host_listings_count,calculated_h ost_listings_count_entire_homes, calculated_host_listings_count_private_roo ms, calculated_host_listings_count_shared_roo ms		
--	---	--	--

Una vez categorizadas, hicimos el análisis correspondiente, observando sus frecuencias, los posibles valores que pueden tomar y sus medidas de resumen.

Tras haber finalizado el análisis inicial de Dataset, observamos las correlaciones y la covarianza. La matriz de correlación demostró que no había fuertes correlaciones en el Dataset, ya sea negativas o positivas. Y sobre todo, que no había correlaciones significativas con price, variable sobre la cual realizamos el análisis predictivo. Hicimos un listado de las correlaciones con price de mayor a menor y la correlación más fuerte era con bathrooms, siendo de solo 0.23. En cuanto a la covarianza, la matriz mostró resultados más elevados, con la mayor covarianza siendo entre id y host_id. Pero eso no quiere decir que fueran relevantes para predecir el precio pues ambos son identificadores únicos, por lo cual tiene sentido que siempre sea un valor distinto del otro y que varíen siempre. Además de esas dos variables, observamos otras que sí tenían covarianza alta y que podrían ser útiles para el análisis.

El siguiente paso fue analizar los datos faltantes del Dataset. Realizamos un gráfico sobre el porcentaje de valores faltantes por variable y notamos que había tres que estaban compuestas 100% por valores NaN, por lo que las eliminamos ya que no eran imputables por nada. El resto de variables, al ser tantas (en este momento 72) imputamos las cuantitativas por mediana y las cualitativas por moda. Continuamos observando los valores atípicos: realizamos boxplots de cada variable para poder observar en más detalle y luego medimos la cantidad de outliers utilizando el IQR (valores fuera del rango intercuartil) y usando el Z-Score con umbral de 3.

Como modelo personalizado, optamos por elegir el K-Nearest Neighbors (KNN), modelo muy sensible a los outliers; por este motivo filtramos los valores atípicos y el Dataset acabó teniendo 18.342 filas.

Entrenamiento y predicción

Para entrenar los modelos, decidimos eliminar algunas columnas que consideramos irrelevantes para la predicción de price. Quitamos:

- **Identificadores y URLs** (como id, scrape_id, listing_url, host_id, host_url, picture_url, host_thumbnail_url, host_picture_url) pues son únicos para cada fila y no representan patrones que puedan aprender los modelos. Además, son enlaces y no nos interesan en este caso.
- **Textos largos o redundantes** (como name, description, neighborhood_overview, host_about, host_name, host_location, host_neighbourhood, host_verifications, bathrooms_text) pues tampoco consideramos que nos ayudara a predecir el precio.
- **La variable auxiliar porcentaje_nulos**, que nos fue útil para el análisis exploratorio pero no corresponde incluirla como feature del modelo.

Además, transformamos algunas columnas con el fin de que puedan ser encodeadas y normalizadas. Las **columnas con fechas** (last_scraped, calendar_last_scraped, first_review, last_review, host_since) fueron convertidas a antigüedad en días. Realizamos esto pues el valor absoluto de una fecha no aporta información al modelo pero, en cambio, la antigüedad (por ejemplo, cuántos días pasaron desde la última reseña o desde que el host se unió) puede ser informativa sobre el estado actual del alojamiento o del comportamiento del anfitrión.

La columna **amenities**, que contiene una lista de servicios como texto, fue transformada a una variable numérica representando la cantidad total de amenities disponibles (amenities_count). Esta simplificación permite evitar una expansión excesiva de columnas si se hiciera one-hot encoding y también aportar un valor que se correlaciona intuitivamente con el precio (más servicios, mayor precio esperado).

También se crearon tres nuevas variables a partir de columnas de texto descriptivo. Estas nuevas variables cuantifican la cantidad de caracteres presentes en las descripciones originales, permitiendo conservar información relevante sobre la extensión de los textos sin necesidad de trabajar directamente con ellos. A continuación, se detallan los cambios:

host_about_length representa la cantidad de caracteres en la descripción proporcionada por el anfitrión sobre sí mismo. Captura de manera numérica la presencia y extensión de la información sobre el anfitrión, lo cual podría correlacionar con la confianza o el profesionalismo percibido.

neighborhood_overview_length indica el número de caracteres en la descripción del vecindario hecha por el anfitrión. Resume la cantidad de detalle proporcionado sobre el entorno del alojamiento, potencialmente relacionado con la calidad o el atractivo del anuncio.

description_length mide la cantidad de caracteres en la descripción principal del anuncio. Permite evaluar de forma numérica cuán extensa es la descripción del alojamiento, información que puede ser indicativa del nivel de detalle o marketing del anuncio

Luego de crear estas nuevas variables, las columnas originales (last_scraped, calendar_last_scraped, first_review, last_review, host_since, amenities, host_about, neighborhood_overview y description) fueron eliminadas del conjunto de datos para evitar redundancia y reducir la dimensionalidad.

Posteriormente, realizamos One-Hot Encoding sobre las variables cualitativas: has_availability, instant_bookable, host_has_profile_pic, host_identity_verified, host_is_superhost, neighbourhood_cleansed. La normalización fue realizada según cada modelo lo necesitara.

Modelo 1: Regresión Lineal Múltiple

En este modelo fue necesario aplicar un proceso de normalización sobre los datos. Una vez normalizados, construimos un regresor lineal siguiendo la metodología vista en clase y procedimos a entrenar el modelo. La performance obtenida fue baja, tanto en el conjunto de entrenamiento como en el de prueba. Para intentar mejorarla, analizamos los coeficientes de la regresión lineal, enfocándonos en los 10 más relevantes. Estos coeficientes reflejan la importancia que el modelo asigna a cada variable: un coeficiente positivo implica que la variable contribuye a aumentar la predicción, mientras que un coeficiente negativo tiende a disminuirla.

Además, realizamos ajustes en el tratamiento de los outliers. Aunque inicialmente los filtramos utilizando un umbral de Z-Score de 4, decidimos hacerlo más estricto y bajarlo a 3. Sin embargo, este cambio no tuvo un impacto positivo en el rendimiento. También intentamos mejorar el modelo revisando el conjunto de datos, eliminando columnas que consideramos poco relevantes y explorando la creación de nuevos features. A pesar de estos esfuerzos, no logramos mejoras significativas.

Finalmente, concluimos que las bajas correlaciones observadas entre las variables disponibles y el precio influyeron negativamente en el desempeño general del modelo.

Modelo 2: XGBoost

En el caso de este modelo, no fue necesario normalizar y solo creamos dummies. Utilizamos K-Fold Cross Validation con 5 folds para mejorar la performance del modelo. También optamos por usar RandomizedSearchCV para hallar los mejores hiperparámetros, ya que es más rápido que GridSearchCV y porque nuestro espacio de búsqueda era grande. Los hiperparámetros que definimos fueron los siguientes:

- `n_estimators`: cantidad de árboles en el bosque (700, 800, 900)
- `max_depth`: controla la profundidad máxima que puede tener el árbol (5),
- `learning_rate`: cada árbol nuevo agrega una corrección y el `learning_rate` define qué tan fuerte es dicha corrección (0.03, 0.04)
- `subsample`: qué porcentaje de los datos ve cada árbol (0.7, 0.8)
- `colsample_bytree`: qué porcentaje de las columnas (features) ve cada árbol (0.7, 0.8)
- `gamma`: divide el árbol según si la mejora es buena (1, 1.5)
- `lambda`: penaliza los pesos de las hojas si son muy grandes (2, 3)
- `alpha`: promueve que algunas hojas tengan peso cero para evitar overfitting (1.0, 1.5)

El desempeño en los conjuntos de entrenamiento y prueba fue mejor con este modelo. Si bien se observó un leve overfitting en el conjunto de entrenamiento, el rendimiento general fue superior al de los modelos anteriores, lo que era esperable dado que se trata de un modelo más potente. Profundizaremos sobre este resultado en la sección de conclusiones.

Modelo 3: K-Nearest Neighbors (K-NN)

Para trabajar con este modelo primero verificamos que no hubiera valores NaN, ya que KNN no los puede manejar. Luego convertimos todas las variables categóricas a numéricas utilizando el One-Hot Encoding. Y, por último, como KNN se basa en distancias (como por ejemplo la euclídea), normalizamos las variables cuantitativas.

Una vez finalizadas las preparaciones iniciales, procedimos a la creación del modelo. Sin embargo, el rendimiento obtenido fue insatisfactorio: se observó un overfitting marcado en el conjunto de entrenamiento y un desempeño extremadamente bajo en el conjunto de prueba, independientemente de los ajustes realizados sobre los hiperparámetros.

Como primera estrategia, implementamos el uso de Pipelining, aunque esto no logró mejorar los resultados. Posteriormente, optamos por la creación de nuevas variables. Si bien esta estrategia no logró corregir el overfitting en el entrenamiento, sí permitió una mejora en el desempeño sobre el conjunto de prueba. La mejora más significativa se reflejó en el valor del R^2 , que pasó de 0,38 a 0,42.

Para la creación de nuevas features tomamos todas las correlaciones del Dataset y vimos cuáles podíamos implementar. Optamos por quedarnos con las siguientes features nuevas:

- **proporcion_listings_completos:** la proporción entre el número de `calculated_host_listings_count_entire_homes` (viviendas completas listadas por el anfitrión) y el número total de posts listados del anfitrión. Indica cuántos de los listados del anfitrión son viviendas completas.
- **proporcion_availability:** la proporción entre la disponibilidad a 30 días y la disponibilidad a 365 días. Indica qué tan disponible está un alquiler en los próximos 30 días en comparación con todo el año.
- **promedio_review_scores:** el promedio de los distintos scores sobre la experiencia de los usuarios (precisión, limpieza, check-in, valor, ubicación y evaluación general). Representa una calificación promedio general del alojamiento basada en diversas dimensiones de la experiencia del huésped.
- **diferencia_review_scores_clean_accuracy:** la diferencia entre las calificaciones de limpieza y precisión de la reseña. Indica si los huéspedes están más enfocados en la limpieza o en la precisión del anuncio.
- **cantidad_bathrooms_bedrooms:** la cantidad de baños por cantidad de dormitorios.

Tras la selección de estas features, medimos las correlaciones de todas las variables con `price`, incluyendo las que habíamos creado. Las correlaciones mejoraron respecto a las que vimos en el análisis inicial gracias a las nuevas features, por lo que nos quedamos con las variables que poseían una correlación mayor o igual a 0.02 y menor a 0 (para incluir a las negativas, que también aportan información).

Luego definimos el KNN, para el cual dividimos los datos usando la transformación logarítmica. Esta toma el valor de la variable y calcula su logaritmo, para reducir la amplitud de las diferencias entre valores grandes y pequeños. Esto lo hicimos porque valores como el precio pueden estar muy sesgados hacia valores pequeños, pero tener unos pocos valores extremadamente grandes y la transformación logarítmica nos ayuda a comprimir estos valores grandes, haciendo que los datos sean más simétricos y fáciles de manejar para el modelo.

En el KNN utilizamos RandomizedSearchCV con 5 folds, realizando una validación cruzada internamente mientras busca los mejores hiperparámetros. Los hiperparámetros que definimos son:

- **n_neighbors**: número de vecinos que el modelo considera para hacer una predicción (3, 5, 7, 9, 11, 13, 15, 17)
- **weights**: cómo se combinan los vecinos para hacer la predicción, cuáles tienen más peso al hacer la ponderación ('uniform', 'distance')
- **metric**: la medida de distancia para saber quiénes son los vecinos más cercanos ('euclidean', 'manhattan', 'chebyshev')

Cuadro de resultados

Medidas de rendimiento del conjunto TEST:

<u>Modelo</u>	<u>MSE</u>	<u>RMSE</u>	<u>R²</u>	<u>MAE</u>
Regresión lineal	663321.02	814.45	0.3467	448.59
XGBoost	441778.32	664.66	0.5649	350.74
KNN	586357.66	765.74	0.4225	388.24

Mejores parámetros encontrados para el XGBoost:

{'subsample': 0.8, 'n_estimators': 900, 'max_depth': 5, 'learning_rate': 0.03, 'lambda': 3, 'gamma': 1, 'colsample_bytree': 0.7, 'alpha': 1.5}

Mejores parámetros encontrados para el KNN:

{'weights': 'distance', 'n_neighbors': 13, 'metric': 'manhattan'}

Significado de las distintas medidas de rendimiento:

- **MSE**: el Error Cuadrático Medio mide el promedio de los errores al cuadrado entre los valores predichos y los valores reales.
- **RMSE**: el Raíz del Error Cuadrático Medio (RMSE) es la raíz cuadrada del MSE. Expresa el error en las mismas unidades que la variable que se está usando.
- **R²**: el Coeficiente de Determinación es la proporción de variabilidad explicada por el modelo. Su valor está entre 0 y 1. Cuando es 0 el modelo se ajusta poco a la variabilidad de los datos. Si es 1 se ajusta mucho a la variabilidad de los datos. Mientras mayor es R², mayor es la fuerza de la/s variable/s regresora/s para predecir la respuesta.
- **MAE**: el Error Absoluto Medio (MAE) mide el promedio de las diferencias absolutas entre los valores predichos y los valores reales.

Analicemos la performance de todos los modelos respecto al set de entrenamiento.

En el caso del modelo de Regresión Lineal Múltiple, el entrenamiento tuvo el siguiente rendimiento:

```
--- Train ---  
MAE: 468.57  
MSE: 738429.01  
RMSE: 859.32  
R2: 0.3710
```

Si lo comparamos con el test, en ambos casos el R^2 es bajo. En test es 0.346. Al ser similares, no nos sugiere que el modelo posea overfitting. Quizás se podría proponer que tiene un poco de underfitting, pero aún así (y comparado al KNN) no consideramos que sea tan grave. También tenemos en cuenta la nula correlación de las variables con price, pues el modelo de Regresión solo tiene en cuenta las relaciones lineales, por lo que su rendimiento malo se debe a eso.

El XGBoost tuvo el siguiente rendimiento en train:

```
--- Train ---  
MAE: 264.05  
MSE: 197073.38  
RMSE: 443.93  
R2: 0.8321
```

En este caso el entrenamiento no tiene tanto overfitting como todos los demás y tiene un MAE menor al de Regresión y mayor al de KNN. Tiene un buen desempeño en el entrenamiento, pudiendo explicar el 83% de la variabilidad de los precios, con un MAE no tan grande. La diferencia con test es lo que nos sugiere un poco de overfitting, ya que el R^2 de test es 0.565, lo cual difiere mucho del train. Aún así, es el que mejor se desempeña de todos los modelos y el MAE del train es de 350, que sigue sin ser excesivo observando el train.

El K-Nearest Neighbors tuvo el siguiente rendimiento en train:

```
--- Train ---  
MAE: 0.03  
MSE: 2.32  
RMSE: 1.52  
R2: 1.0000
```

En el conjunto de entrenamiento se aprecia un overfitting perfecto, sobre todo por el R^2 que es igual a 1. En cambio, en el conjunto test se aprecia un R^2 de 0.42,

que es aceptable considerando la baja correlación inicial. Esto quiere decir que el modelo puede explicar el 42% de la variabilidad de los precios. Además, el MAE nos dice que el modelo se equivoca en aproximadamente 388 dólares por predicción. Por lo tanto, no es el mejor modelo y lo peor es el train, pero no hallamos una alternativa para disminuir el overfitting del train y subir el rendimiento de test.

Luego de nuestro análisis del dataset en esta ocasión, concluimos que elegiríamos XGBoost para predecir el precio de un alquiler de AirBnB en la ciudad de Río de Janeiro, pues fue el que mejor rendimiento tuvo en test y en train. Es el mejor de los tres en lograr capturar patrones para la predicción del precio y es el que posee un MAE menor y más razonable. Además su RMSE es moderado en comparación a los otros dos modelos, lo que refuerza su elección como el modelo más robusto.

Cuadro de tiempo dedicado

Integrante	Tarea	Prom. Hs Semana
Ramiro Barcala Roca	Implementación del XGBoost	6
Rafaela Pilar Arias	Implementación de Regresión Lineal Análisis inicial de variables Armado del reporte	8
Mora Lema	Análisis inicial de variables Implementación del KNN Armado del reporte	8
Lara Eliana Ienco	Implementación de Regresión Lineal	4